

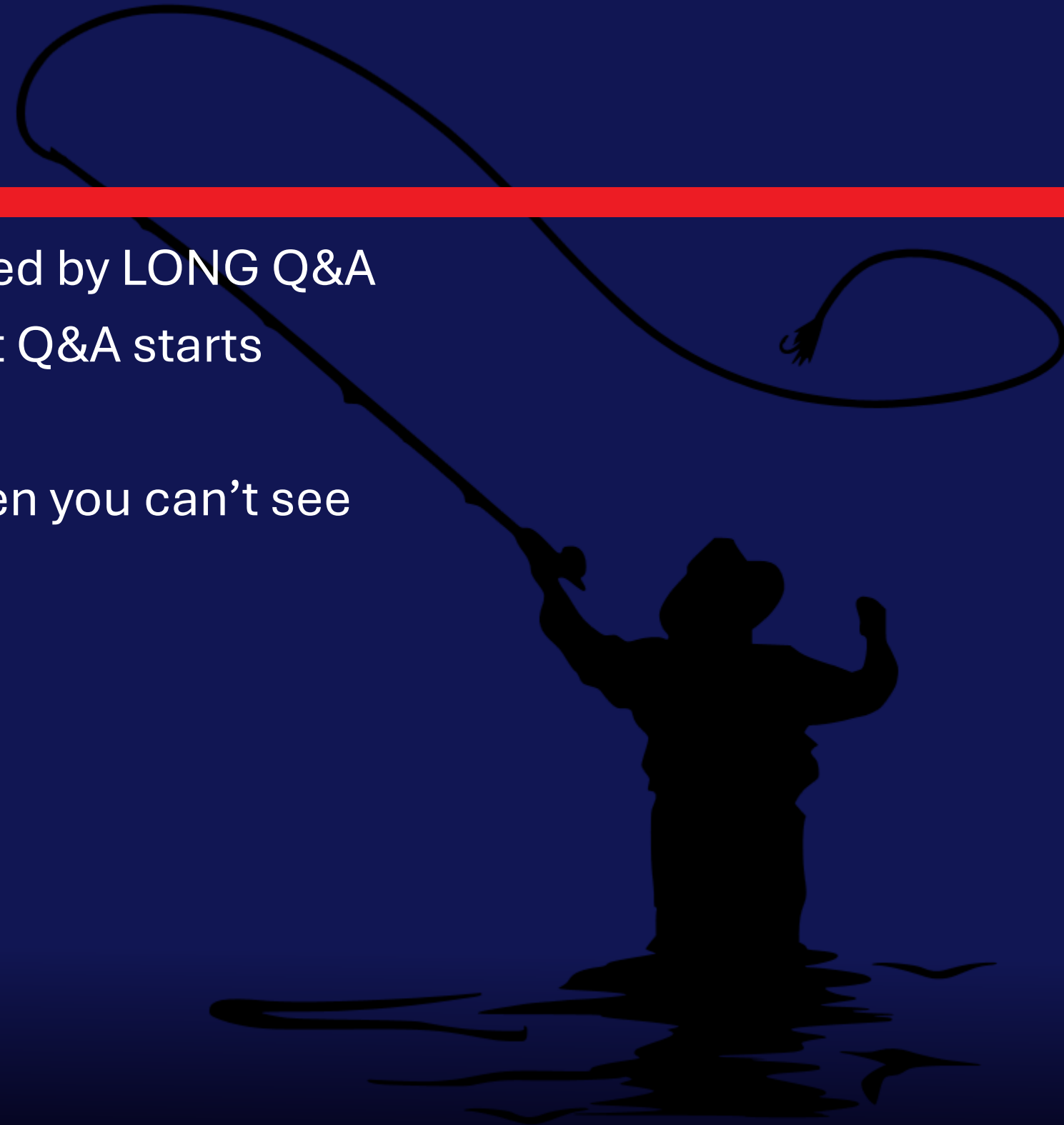
Cure Script Fatigue

Reliable Endpoint State with DSC v3



Attention

- MMS sessions are 60-75 minutes followed by LONG Q&A
- Please hold detailed questions until that Q&A starts
- Feel free to ask clarification questions
- Remind the speakers to use Zoom It when you can't see





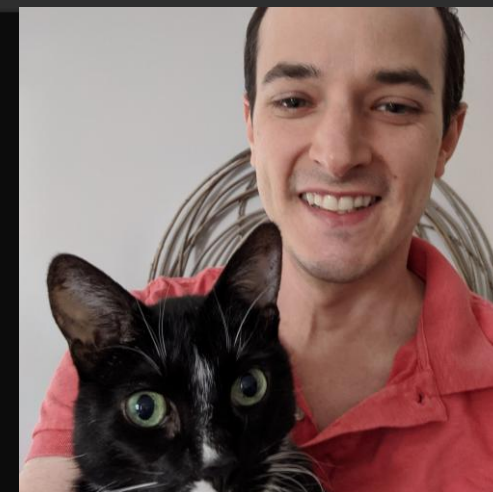
Command Prompt



```
C:\Users\flesniak>whoami
Frank Lesniak (@franklesniak)
Tech enthusiast; Microsoft 365 consulting team lead at West Monroe
```

```
Experience:
Almost 20 years in consulting. PowerShellin' since 2006. Focused on corporate M&A.
```

```
Credentials:
MCSEs. Co-leads Chicago PowerShell Users Group. PowerShell + DevOps Global Summit speaker alumnus.
```



```
*****
```

```
Ask Me About:
```

- PowerShell automation (duh!)
- Executing corporate divestitures and integrations - where I spent most of my consulting client time
- My AI-coding (vibe coding) harness
- Retro video game software and FPGA emulation
- My upcoming home construction project

```
*****
```

```
Contact:
```

- x.com/franklesniak
- bsky.app/profile/franklesniak.com
- linkedin.com/in/flesniak
- github.com/franklesniak
- infosec.exchange/@franklesniak
- flesniak ATSIGN westmonroe.com



```
C:\Users\flesniak>|
```

```
C:\Users\bcherry>whoami
Blake Cherry (@blakelishly)
Enterprise Technology Principal at West Monroe
```

Experience:
5 yrs. in consulting, focused on supporting divestitures and integrations through automation and cloud/on-premises infrastructure solutions.

Credentials:
Lead for West Monroe's greenfield environment build automation tooling.
West Monroe's Infrastructure Automation Interest Group lead.
PowerShell + DevOps Global Summit alum

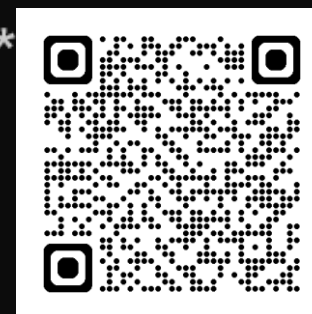
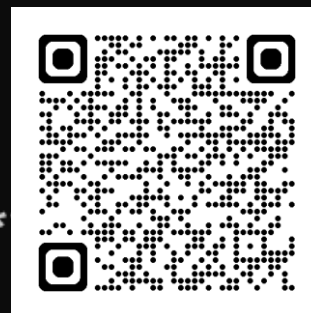


Ask Me About:

- PowerShell and Infrastructure as Code Automation
- My homelab and self-hosted services
- DevOps and Pipeline Automation
- Azure Landing Zones

Contact:

- [linkedin.com/in/rblakecherry](https://www.linkedin.com/in/rblakecherry)
- github.com/Blakelishly
- [bcherry AT SIGN westmonroe.com](mailto:bcherry@westmonroe.com)



```
PS C:\Users\bcherry> |
```

What You Will Walk Away With

Two-Layer Operating Model

Adopt a model that uses Intune/ConfigMgr for delivery while leveraging DSC v3 for reliable, self-correcting configuration state.

Production-Grade Execution Template

Standardize exit codes, prevent "false green" reporting, and capture durable evidence for troubleshooting.

Understand Practical Techniques

Master practical techniques for offline packaging, reboot orchestration, and secrets management to replace brittle imperative scripts with reliable DSC v3 patterns.

GitHub Starter Kit

Walk away with a complete starter kit containing runner templates, validation tools, and sample configurations ready to deploy in your environment.

Show of Hands

The Silent Failure

Have you shipped a remediation script that reported success while the endpoint stayed wrong?

The Proof Gap

Have you needed proof of compliance, but the only answer available was "the script said it ran"?

If either of these is familiar, this talk is for you.

False Green Is Success You Cannot Trust

False green means the management plane recorded success while the endpoint stayed wrong. The user still sees the problem. The audit still finds the drift. The engineer still has to prove what happened.

Management Plane Says

- ✓ Success — script executed, exit 0 returned, portal status green.

What is Really Going on with the Endpoint

- ✗ State unchanged. Drift persists. No durable evidence was written.



Script Fatigue Is a Proof Problem

Endpoint teams already have scripts.

They need a dependable way to prove desired state, translate outcomes, preserve evidence, handle reboots, and rerun safely.

When proof is weak, every platform has to trust a thin success signal and a few lines of output.

Prove State

Confirm the endpoint actually reached desired state.

Preserve Evidence

Write durable output that survives portal reporting.

Rerun Safely

Guarantee idempotent behavior across repeated executions.

Translate Outcomes

Surface structured exit codes the execution plane can act on.

Handle Reboots

Signal reboot requirements to the plane that owns restart policy.

DSC v3 in One Minute

What is DSC v3?

DSC v3 is a standalone, cross-platform CLI for Desired State Configuration. It evaluates JSON or YAML configuration documents, invokes resources, tests desired state, sets state when asked, and emits structured output.



Another tool still decides when and where DSC v3 executes. DSC v3 owns state logic, not scheduling.

Configuration Document

JSON or YAML —
declares desired state

dsc.exe

Invokes resources · tests
state · sets state

Structured Output

JSON result — parseable, durable evidence

DSC v3.2.0 Is the Current Stable Baseline

DSC v3.2.0 GA was announced April 29, 2026.

**Built-in Windows
Resources**

Version Pinning

**Richer
Expressions**

Custom Functions

**Adapter
Improvements**

Secret Extension

**Discovery
Extension**

**Bicep over gRPC
(Experimental)**

How DSC Tooling Changed

Earlier PowerShell DSC

- Configuration scripts compiled to MOF
- LCM settings managed locally
- Push or pull workflow models
- Pull server infrastructure
- Tightly coupled to PowerShell engine

DSC v3

- Standalone cross-platform CLI
- JSON or YAML configuration documents
- Resource adapters for existing resources
- Structured JSON output on every run
- No LCM, no MOF, no pull server required

The desired-state idea carries forward. The architecture is substantially different.

DSC v3 is Open Source and Owned by the PowerShell Team, but is a Seperate Product

Product Home

Find and track new DSC updates in the DSC repository or from PowerShell Team release posts.

Standalone Runtime

The runtime is a standalone `dsc` executable, and the open-source repo is primarily Rust.

No PowerShell Dependency

DSC v3 does not depend on PowerShell, Windows PowerShell, or the `PSDesiredStateConfiguration` module.

Adapter Integration

When PowerShell is present, DSC v3 can invoke PowerShell DSC resources through adapter resources.

Integration Path, Not Engine

PowerShell is an integration path, not the required engine.

You can still use DSCv3 and PowerShell together <3

A Configuration Document Declares Correct State

Document Structure

A DSC v3 configuration document is a JSON or YAML object containing `$schema`, `resources`, and optional `metadata`, `parameters`, `variables`, and `directives`. DSC validates the document, invokes each resource instance, and returns structured results.

Each Resource Instance Has

- A unique `name`
- A `type` referencing the resource
- A `properties` block declaring desired state

`$schema`

Identifies document version and structure

`resources[]`

Array of resource instances with name, type, properties

`metadata / parameters`

Optional context, variable substitution

Structured Result

JSON output per resource, per run

Use YAML and JSON Deliberately

configs/baseline.dsc.yaml

```
$schema:
https://schemas.microsoft.com/dsc/2024/04/config.schema.json
metadata:
  description: ProStateKit baseline
resources:
- name: Baseline-ControlledLocal
  type: Microsoft.Windows/Registry
  properties:
    keyPath: HKLM\SOFTWARE\ProStateKit
    ensure: Present
```

Author in YAML when humans need to review, approve, or diff the baseline.

configs/generated/baseline.dsc.json

```
{
  "$schema":
    "https://schemas.microsoft.com/.../config.schema.json",
  "metadata": {
    "description": "ProStateKit baseline"
  },
  "resources": [
    {
      "name": "Baseline-ControlledLocal",
      "type": "Microsoft.Windows/Registry",
      "properties": {
        "keyPath": "HKLM\\SOFTWARE\\ProStateKit",
        "ensure": "Present"
      }
    }
  ]
}
```

Generate JSON for fixtures, schema validation, and exact comparisons.

Parse captured DSC command output as JSON. Do not maintain two hand-authored versions.

The Four DSC v3 Moves We Need



1. Define

Declare desired endpoint state in a JSON or YAML configuration document.



2. Test

Run `dsc config test` to check whether the endpoint matches declared state.



3. Set

Run `dsc config set` to converge the endpoint to declared state when drift is detected.



4. Parse

Read structured JSON output per resource — exit code alone is not sufficient proof.

Native First, DSC When State Needs Proof

Use Native Controls When

- The requirement maps directly to a first-party setting
- The platform provides built-in reporting
- Fewer moving parts are a real advantage
- User experience and restart policy are owned by the plane

Use DSC v3 When

- State needs reviewed, version-controlled definition
- Repeatable test and set behavior is required
- Structured output must survive portal reporting
- Idempotent reruns are a safety requirement
- Durable evidence is a compliance requirement

The Handoff That Prevents False Green



Execution Plane

Owens target, schedule, identity, transport, portal

Wrapper Script

DSC invocation, parsing, evidence

DSC v3 Engine

Configuration doc, test/set results, structured output

The execution plane starts the run. The runner mediates. DSC v3 proves state. Evidence ties all three together into a durable, inspectable record that survives portal output.

The Execution Plane Owns Reach and Context

Plane	Ownership	Typical Use
Intune	Fleet assignment, portal reporting	Windows endpoints, Remediations
ConfigMgr	Fleet management, compliance settings	Co-managed or standalone Windows
Arc / Machine Config	Azure governance, server audit	Server and hybrid targets
CI Pipeline	Build-time validation gate	PR validation, bundle testing
Scheduled Task	Local recurrence	Lab, break-glass, troubleshooting
PSEXEC / SSH	Operator reach	Lab, break-glass only

State Logic Is Defined in the DSC v3 Payload

DSC v3 Payload Owns

- Declaring correct state
- Testing whether the endpoint matches
- Converging state when asked
- Emitting structured output per resource

Execution Plane Still Owns

- Assignment and targeting
- Scheduling and retry cadence
- User impact and notifications
- Portal status and reporting
- Restart policy and maintenance windows

 The split is intentional. DSC v3 does not replace the execution plane — it strengthens the evidence the execution plane reports.

The Pattern Travels, the Runner Rules Change

The same test → set → evidence idea applies across platforms. Packaging, identity, exit codes, logs, and reporting differ.

Platform	Execution Surface	Notes
Windows endpoints	Intune Remediations, ConfigMgr Compliance	Primary focus
Windows Server	Arc, Azure Machine Config, ConfigMgr, CI, SSH	Plane varies by governance model
Linux / Server	Arc, CI, SSH, systemd timers	Resources and packaging differ
macOS	Apple MDM/DDM, Jamf, nix-darwin	Stay native-first; DSC v3 can run but is not the default plane

The DSC Executable Is a Dependency You Must Distribute

Distribution Modes

Bundled with Payload

ZIP artifact bundled with the payload. Version and hash validated at runtime. Production-safe.

Already Installed on Endpoint

Governed installation, approved path. Version and hash validated at runtime.

Download on Demand

Orchestrated dynamic download of core DSC executable.

Evidence Records and Requisite Details

- Runtime source path
- Version string
- Expected hash
- Observed hash
- Validation result



Do not assume `dsc.exe` is already on the endpoint.
No silent live download in production runs.



ProStateKit
by OpsCura

Healthy endpoints. Happy auditors.

Prove it. Every time.

prostatekit-repo

prostatekit-repo

.prostatekit

polices

baseline.yaml

evidence

2025-05-16

2025-05-15

reports

compliance-summary.json

endpoint-inventory.csv

assets

laptops

servers

config

runtime

packages

README.md

LICENSE

COMPLIANCE

✓

All policies passing

42 / 42 checks

EVIDENCE

📁

2,842 evidence files

Collected & verified

RUNTIME

📦

12 runtimes

Up to date

PACKAGES

📦

236 packages

No known CVEs

ENDPOINTS

🖥️

1,375 endpoints

Healthy

Endpoint state

Compliant

As of May 16, 2025 7:42 AM

Evidence

2,842 files

Collected & verified

Runtimes

12

Up to date

Packages

236

No known CVEs

Endpoints

1,375

Healthy

prostatekit scan --all --prove

Scan complete. No issues found.

ProStateKit by OpsCura

Enterprise endpoint state. Continuous proof. Zero surprises.

CONTINUOUS
VISIBILITY

AUTOMATED
COMPLIANCE

AUDITOR
READY

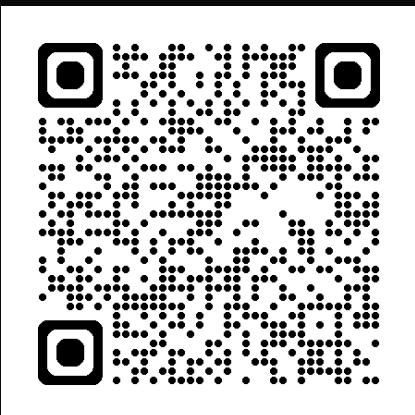
OPS FUSION 2025

TECHNOLOGY. TRUST. TOMORROW.

PROSTATEKIT

Enter ProStateKit: A Starter Kit for DSC v3 Payloads

ProStateKit packages runner behavior, validation gates, plane shims, and durable evidence into one reviewed, version-controlled artifact. It is not a Machine Configuration package — it is an opinionated starting point.



GitHub Repo:

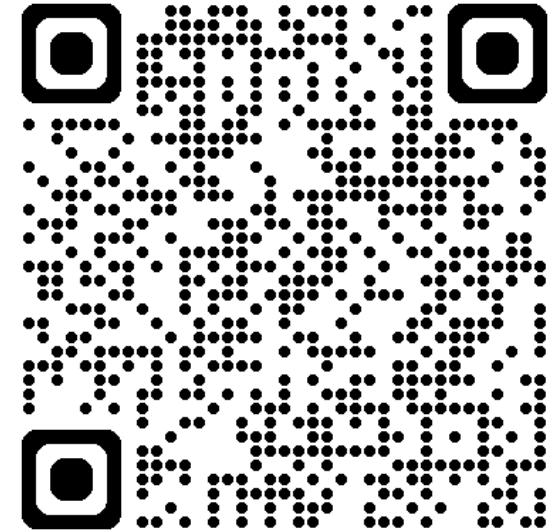
This is a parody advertisement for a fictional product.
ProStateKit by OpsCura is not a real product. Not intended to diagnose, treat, cure, or prevent any security issues.

ProStateKit **Pk**
DSC

Relief for script fatigue

Test • Set • Verify • Report

ASK YOUR ARCHITECT



Link to Video

ProStateKit

Ask your architect if ProStateKit is right for you.

Reliable endpoint state with DSC V3



Issues...? ProStateKit Can Help!

Repository Structure

```
src/  
  Invoke-ProStateKit.ps1  ← public entry point  
  runner/Runner.ps1      ← common runner  
planes/  
  intune/  
  configmgr/  
  local/  
configs/  
  baseline.dsc.yaml  
  generated/baseline.dsc.json  
schemas/  
docs/  
evidence/
```

What the Structure Enables

The execution plane receives known inputs and writes comparable evidence. Every run can be traced back to a source commit and a validated artifact.

Reviewed Artifact

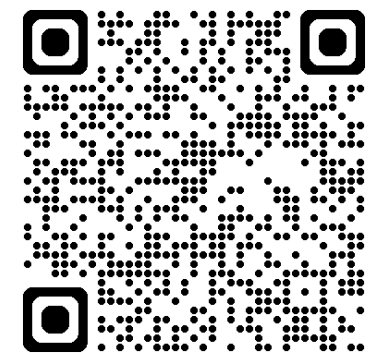
One bundle per plane

Known Inputs

Stable and consistent parameter inputs

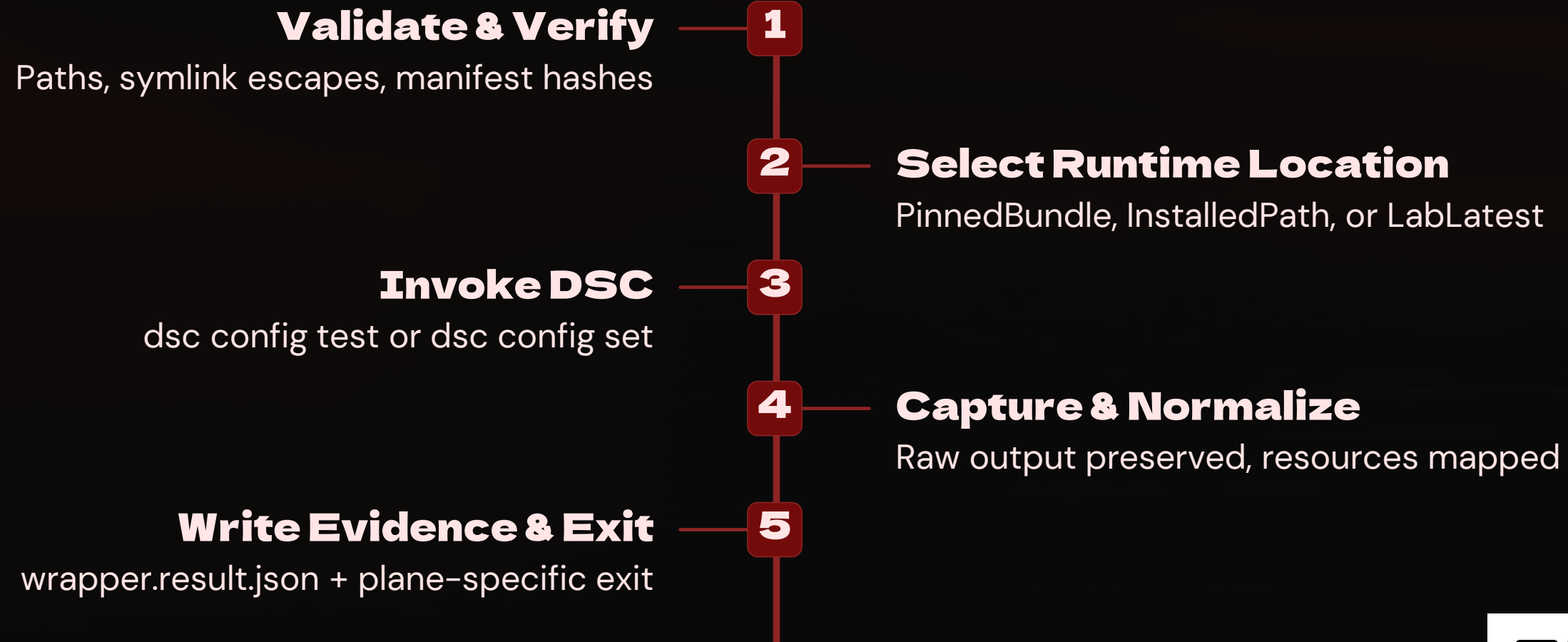
Comparable Evidence

Same schema across planes



GitHub Repo:

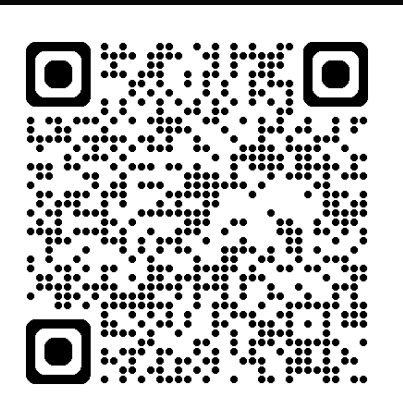
The Runner Gives Every Plane the Same Interface



Raw DSC output stays preserved.

Automation reads `wrapper.result.json`.

GitHub Repo:



Stable Inputs Make Runs Repeatable

```
pwsh -File .\src\Invoke-ProStateKit.ps1 `
  -Mode          Detect | Remediate | ValidateBundle | Preflight `
  -ConfigPath     .\configs\baseline.dsc.yaml `
  -RuntimeMode    PinnedBundle | InstalledPath | LabLatest `
  -RuntimePath    <path to dsc.exe> `
  -RuntimeExpectedHash <sha256> `
  -RuntimeExpectedVersion <semver> `
  -EvidenceRoot   <path> `
  -Plane          Local | Intune | ConfigMgr | CI `
  -BundleRoot     .
```

 Parameters are validated before any DSC invocation. Missing or mismatched values fail closed — the run never reaches the endpoint without verified prerequisites.

Detect Is a State Check



Run DSC Test

Parse JSON

Write Evidence

Return Exit

What Detect Does

Detect mode runs `dsc config test` with JSON output format. The Runner captures stdout and stderr, parses the structured result, writes evidence, and returns a plane-safe exit.

Exit Outputs

- 0 — Compliant (Intune-safe green)
- 1 — Drift or proof failure
- Detail always stays in `wrapper.result.json`

Remediate Sets State, Then Proves It



1. dsc config set

Converge endpoint to declared state. Capture raw set output.



2. dsc config test

Verify convergence. Run a full test after the set attempt.



3. Write Evidence

Capture both raw artifacts and the normalized wrapper result.



4. Return Exit

Success only if post-set verification proves convergence. Failure stays red.

Running `dsc config set` without a follow-up test is not Remediate — it is a hope. The Runner always verifies before returning green.

Every Run Leaves Evidence

Evidence Folder Contents

```
evidence/{runId}/
  dsc.test.stdout.raw.json
  dsc.set.stdout.raw.json      # Remediate only
  dsc.stderr.txt
  exit-code.txt
  runtime-metadata.json
  manifest-snapshot.json
  wrapper.result.json         # normalized decision
  run-summary.txt
  runner.log
  reboot.marker.json          # if rebootRequired
Current/
  last-result.json            # latest normalized decision
```

Why Every Run Writes

A green run still writes evidence. Compliance without a record is an assertion without proof. Recurring runs build an audit trail. Evidence survives portal output and script transcript rotation.



Current/last-result.json always reflects the most recent normalized decision — compliant or not.

Exit Codes Differ Layer-by-Layer

Layer	Code	Meaning
DSC	0	Command completed without runtime error
DSC	Non-zero	Runtime error — not a state judgment
Runner	0	Compliant — state matches declaration
Runner	1	Non-compliant — drift detected
Runner	2	Runtime failure — DSC did not complete
Runner	3	Parse or proof failure — evidence invalid
Intune	0	Compliant (Detect) / Success (Remediate)
Intune	1	Non-compliant — triggers Remediate

The public entry point maps non-green Runner outcomes to plane-safe exits. Detail always stays in evidence.

Missing Proof Fails Closed



⊗ No proof, no green. An unknown output shape, missing evidence file, or hash mismatch returns non-compliant. The Runner never infers success from silence.

Secrets Are Designed Before Evidence Capture

Where Secrets Must Not Appear

- Configuration documents
- Scripts and transcripts
- Raw DSC stdout or stderr
- Normalized evidence files
- Screenshots and terminal captures
- Platform portal output

Safer Patterns

Platform Identity

Managed identity or device credential — no embedded values

Resource-Native Secure Input

Resources that accept secure strings natively, not raw config properties

Evidence by Reference

Record references and outcomes — never secret values

DSCv3 is not the Best Tool for Complicated Software Distribution or Patching

App Deployment and Patching Tools Own

- Broad software distribution
- Emergency patching and supersedence
- User experience and notifications
- Restart orchestration
- Catalog and license management

DSC v3 Is Appropriate For

- Declared package state with pinned source
- Internal feed or approved artifact
- Deterministic detection logic
- Idempotent install behavior
- Exit translation and durable evidence



Do not turn a DSC resource into an unmanaged package manager. If the requirement is broad distribution or emergency patching, reach for the right tool.

Version Control Makes Endpoint State Reviewable



Source Commit

Pull Request

Bundle Artifact

Endpoint Evidence

Version control gives endpoint state a change history, review path, rollback point, and release identity.

The bundle ties endpoint evidence back to a source commit and a validated artifact. Drift investigations start with a known-good baseline, not a transcript.

Linting Catches Problems Before Upload

What the Gate Checks

- Markdown structure and link validity
- DSC configuration schema lint
- PowerShell parse and PSScriptAnalyzer checks
- Pester test suite (**99 tests**)
- Release-readiness check against external evidence

Run Locally

```
npm run validate -- -SkipPreCommit
```

The same validation gate runs in CI. Problems caught locally do not reach the PR gate.



Release-readiness checks keep the bundle tied to external evidence before promotion to production.

Agents Can Accelerate Changes, CI Owns the Gate

Agents can help with issue triage, branch setup, small-scope edits, test scaffolding, and doc updates. Humans own correctness, safety, and production rollout. CI owns the gate between authoring and promotion.



Authoring & Local Validation

- Agent-assisted edits, branch setup, test scaffolding
- Local preflight and lint pass



CI & Human Review

- Automated gate: schema, hash, lint, DSC dry-run
- Human approval required before promotion



Lab Runs & Production Approval

- Lab execution with evidence capture
- Human sign-off owns production rollout

Local Preflight Proves the Bundle

```
pwsh -File .\planes\local\Invoke-LocalPreflight.ps1 -BundleRoot . -RuntimeMode PinnedBundle
```

What Preflight Validates

1 Bundle Metadata

Manifest version, hash, and integrity checks

2 Runtime Prerequisites

DSC executable version and hash validation

3 Known-Good Detect

Green run before introducing drift

4 Drift → Remediate → Final Detect

Full cycle gate before upload

Preflight is the same gate used for demo rehearsal, CI validation, and pre-upload verification. If preflight fails, the bundle does not get promoted.

```
PS C:\ProgramData\ProStateKit\Bundle> & '.\planes\local\Invoke-LocalPreflight.ps1' -BundleRoot . -RuntimeMode PinnedBundle
Bundle validation completed successfully.
{
  "mode": "PinnedBundle",
  "path": "C:\\ProgramData\\ProStateKit\\Bundle\\runtime\\dsc\\dsc.exe",
  "version": "dsc 3.2.0",
  "source": "BundleRoot",
  "expectedHash": "TBD",
  "observedHash": "sha256:6ee88bd4c93c4a94539a0af0667ace8ffba48f5b8732930e1421721621ca19de"
}
PS C:\ProgramData\ProStateKit\Bundle> _
```



Enterprise endpoint state.
Proven. Observable. Trusted.

Compliance Posture
98% Compliant

Runtime & Packages
Up to date

Evidence Collected
42 Items

Security Aligned
Zero critical

Endpoints Healthy
1,384 / 1,412

Policy Enforced
Drift: 0

```
Administrator: ProStateKit
> ProStateKit scan: FINISHED
> Endpoint: LAPTOP-7Q3K2M1
> Time: 2025-05-20 07:31:24

[OK] OS Version           Windows 11 Pro 23H2
[OK] BitLocker             Enabled
[OK] Firewall              Active (Domain)
[OK] Antivirus             Active (Healthy)
[OK] Disk Encryption       Compliant
[OK] OS Patch Level        Up to date
[OK] Secure Boot           Enabled
[OK] TPM                   Detected 2.0
[OK] Power Plan            Balanced
[OK] UEFI Firmware         Up to date
[WARN] Third-Party AV      Contoso AV detected
[WARN] Local Admins        2 found
[OK] Endpoint Config       Compliant

> Overall State: GOOD
```

LIVE DEMO

Break a Windows Endpoint. Detect Drift. Repair It. Prove It.

The next six steps use a controlled endpoint, a version-controlled bundle, and a pinned DSC v3 runtime. Every result is written to evidence before we call anything green.



For IT operations and systems management professionals. Results may vary.
ProStateKit is a fictional product. Not intended to diagnose, treat, cure, or prevent any IT problems.

Six Key Demo Points

01

Verify Bundle and Runtime

Confirm bundle integrity, runtime version, and hash before any DSC invocation.

03

Introduce Deterministic Drift

Run the scripted drift tool. Break the exact state the baseline covers.

05

Remediate and Verify

Run Remediate. Runner sets state, then verifies convergence with a follow-up test.

02

Known-Good Detect

Run Detect on a clean endpoint. Inspect green evidence and confirm classification is Compliant.

04

Detect After Drift

Run Detect on the broken endpoint. Confirm red is correct — classification is NonCompliant.

06

Final Detect

Run Detect one more time. Inspect the evidence record. Confirm idempotent green.

Demo Targets

Component	Value
Target endpoint	Controlled Windows endpoint — lab environment
Configuration	<code>configs/baseline.dsc.yaml</code>
Resource: Local group	<code>Baseline-ControlledLocal</code> — group membership declared
Resource: Registry	<code>EnableMulticast</code> — LLMNR registry key disabled
Resource: Marker file	<code>.\ProStateKit\Baseline\baseline-applied.txt</code>
Success condition	Endpoint state, DSC result, ProStateKit result, and evidence all agree

 All three resource types are present in the baseline to give a meaningful cross-section of detection and remediation behaviors.

Known-Good Detect Produces Inspectable Green

```
pwsh -File .\src\Invoke-ProStateKit.ps1 `
  -Mode      Detect `
  -Plane     Local `
  -ConfigPath .\configs\baseline.dsc.yaml `
  -RuntimeMode PinnedBundle `
  -BundleRoot .
```

Expected Results

- Exit code: 0
- classification: Compliant
- Raw DSC stdout captured to evidence
- Runtime metadata and manifest proof preserved

Deterministic Drift Makes Red Meaningful

```
pwsh -File .\src\tools\New-DemoDrift.ps1
```

Target	Before	After Drift
Controlled local group	Members match declared state	Member removed
LLMNR registry setting	Key disabled as declared	Key deleted or re-enabled
Baseline marker file	File present at declared path	File deleted

The break is repeatable, reversible, and visible in evidence. Scripted drift removes the risk of a missed manual step during the demo.

Red Is Correct When Drift Exists

Detect After Scripted Break

- Exit code: 1
- `classification: NonCompliant`
- Raw DSC output captured
- Resource-level drift recorded in `wrapper.result.json`
- Reference evidence: `evidence/sample/noncompliant-detect`



Red here is not a failure of the system — it is proof the detection works. Red when drift exists is the correct behavior.

Remediate Writes Set and Verification Evidence

```
pwsh -File .\src\Invoke-ProStateKit.ps1 `
  -Mode      Remediate `
  -Plane     Local `
  -ConfigPath .\configs\baseline.dsc.yaml `
  -RuntimeMode PinnedBundle `
  -BundleRoot .
```

Evidence Written

- `dsc.set.stdout.raw.json` — raw set result
- `dsc.test.stdout.raw.json` — post-set verification
- `wrapper.result.json` — normalized decision

The Runner returns success only after post-set verification proves convergence. Set without verification is not Remediate.

Idempotence Proves Safe Reruns

Final Detect Results

- Exit code: 0
- classification: Compliant
- No corrective action needed
- New evidence record written

The second green matters. Recurring execution by the management plane should not create new drift. A safe rerun proves the pattern holds beyond the first pass.



Idempotence is a design requirement, not an assumption. The final Detect confirms it.

The Demo Proved Test, Set, Evidence

Detect

Runs `dsc config test`. Returns red when state is wrong. Returns green with evidence when compliant. Never infers.

Remediate

Runs `dsc config set`, then verifies with another test. Returns green only after convergence is proven.

Evidence

Captures raw DSC output and the ProStateKit decision. Both survive the run. Both are inspectable.

Agreement

Endpoint state, DSC result, Runner result, and evidence file all agree. That is the proof chain.

Intune Remediations Have Specific Limits

Intune Shim Locations

```
planes/intune/Detect-ProStateKit.ps1  
planes/intune/Remediate-ProStateKit.ps1
```

Shim Behavior

- Calls the common entry point
- Uses PinnedBundle exclusively
- Rejects LabLatest
- Returns short summaries within portal output limits
- Full proof stays on disk in evidence folder



Intune Remediations has a script output size limit.
Short summary to portal — full evidence to disk.
Never write raw DSC JSON directly to portal output.

ConfigMgr Reuses the Payload and Changes the Surface

ConfigMgr Shim Locations

```
planes/configmgr/Discover-ProStateKit.ps1  
planes/configmgr/Remediate-ProStateKit.ps1
```

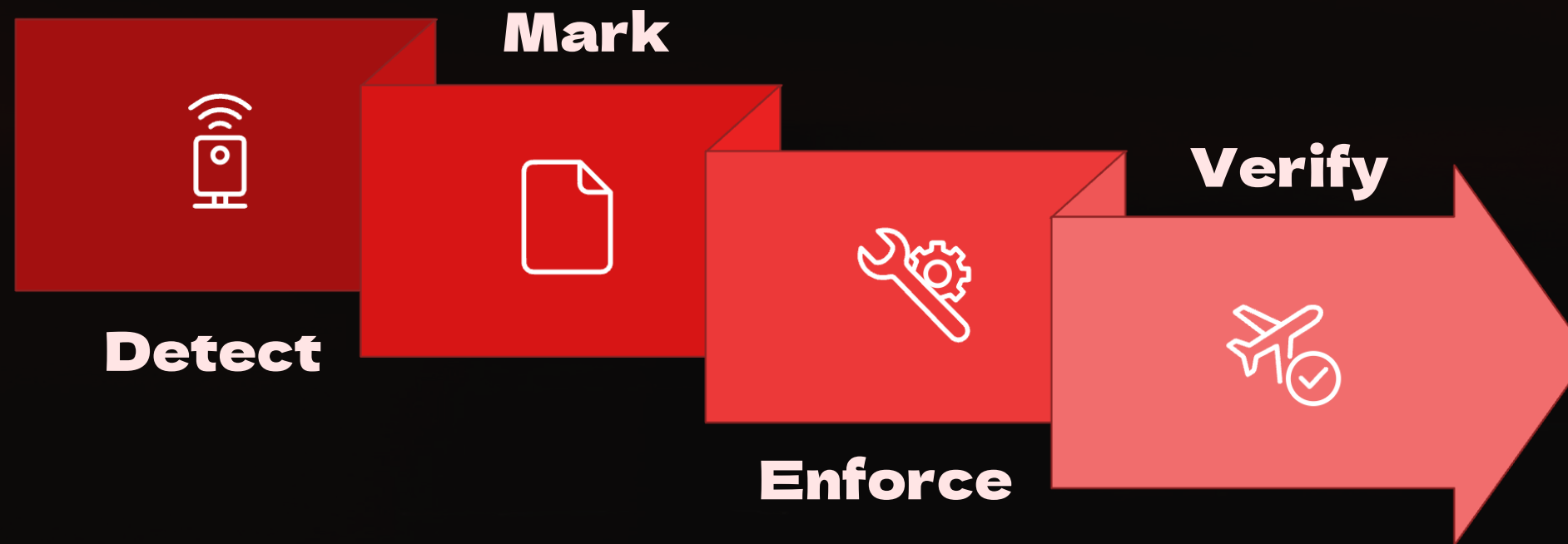
Conversion Notes

- Calls the common entry point
- Outputs **Compliant** or **NonCompliant** as discovery string
- Application deployment places the bundle
- Compliance Settings evaluates state via discovery script
- Same evidence schema as all other planes



The payload is the same. The surface — discovery script output type, run context, 64-bit behavior, log location — is what changes for ConfigMgr.

The Execution Plane Owns Reboots



Reboots are state transitions with user impact. DSC v3 exposes reboot-related signals. The execution plane owns what happens next.

The Runner writes `reboot.marker.json` when normalized proof reports `rebootRequired`. Post-reboot rerun proves the endpoint reached desired state.

What else can you do with what you have learned today?



Reboot Orchestration

Plane-managed continuation, marker TTL, and audit requirements.



Central Reporting

Aggregating evidence across fleet runs into a queryable state store.



Azure Machine Config

Packaging for Arc and Azure governance paths after core proof is established.



Secrets Extensions

Platform identity, managed identity, and resource-native secure inputs.



Agent-Assisted Authoring

AI-assisted policy authoring with CI gates and human review checkpoints.



macOS / Linux Variants

Native-first macOS story and Linux/server plane variants.

You do not need all of that to start. You need one state definition, one runner, and evidence you trust.

Good Endpoint State Has Proof

Declared State

Version-controlled configuration in `baseline.dsc.yaml` with a change history and review path.

Verified Runtime

Pinned DSC v3 executable, hash-verified, recorded in evidence before invocation.

Detect and Remediate

Detect as a real state test. Remediate as set plus verification — not set and hope.

Durable Evidence

Evidence that survives portal output rotation and audit requests.

Understood Exits

Exit codes the execution plane can act on, translated from DSC and Runner semantics.

Repeatable Runbook

A runbook that can prove the pattern again next week, by another engineer, on a different endpoint.

Replace One Remediation Script Next Monday

1 Pick One Script

Choose a remediation script that might be better expressed as declared state — one with a known detection rule and a clear desired outcome.

2 Write It Down

Desired state. Detection rule. Remediation action. Evidence needed afterward. Exit behavior the execution plane expects.

3 Decide

Does DSC v3 own that state better than the current script? Does the proof gap justify the investment?

4 Start with One Technical Problem

Not the entire remediation library. One definition, one runner, one evidence record. That is the start.

What You (Hopefully) Are Walking Away With

Two-Layer Operating Model

Adopt a model that uses Intune/ConfigMgr for delivery while leveraging DSC v3 for reliable, self-correcting configuration state.

Production-Grade Execution Template

Standardize exit codes, prevent "false green" reporting, and capture durable evidence for troubleshooting.

Understand Practical Techniques

Master practical techniques for offline packaging, reboot orchestration, and secrets management to replace brittle imperative scripts with reliable DSC v3 patterns.

GitHub Starter Kit

Walk away with a complete starter kit containing runner templates, validation tools, and sample configurations ready to deploy in your environment.

SAVE THE DATES

Oct 25-28, 2026



May 2-6, 2027



Oct 10-13, 2027



Extended Q&A



Release Notes to Recheck Before Deck Freeze

Latest Stable Release

Confirm version string matches pinned bundle. Update evidence expectations if release changed.

Patch Availability

Check whether a patch release landed between authoring and delivery.

Artifact Hashes

Re-verify expected hashes match published checksums for pinned artifacts.

Built-In Windows Resources

Confirm resource list and property shapes are current for the pinned version.

`secret()` Extension

Review behavior changes and execution context requirements.

PowerShell Adapter Notes

Check adapter compatibility for any PowerShell DSC resources used in demo baseline.

Version-Pinning Syntax

Confirm syntax for specifying exact version in bundle and configuration documents.

YAML and JSON Example Pair

Both files represent the same three baseline resources. YAML for review and authoring; JSON for generation, schema validation, and programmatic comparison.

configs/baseline.dsc.yaml

```
$schema: https://schemas.microsoft.com/dsc/2024/04/config.schema.json
metadata:
  description: ProStateKit baseline
resources:
  - name: Baseline-ControlledLocal
    type: Microsoft.Windows/Group
    properties:
      groupName: ProStateKit-Controlled
      ensure: Present
  - name: EnableMulticast
    type: Microsoft.Windows/Registry
    properties:
      keyPath: HKLM\SOFTWARE\Policies\Microsoft\Windows NT\DNSClient
      valueName: EnableMulticast
      valueData: 0
      ensure: Present
  - name: BaselineMarkerFile
    type: Microsoft.Windows/File
    properties:
      destinationPath: C:\ProgramData\ProStateKit\Baseline\baseline-applied.txt
      ensure: Present
```

configs/generated/baseline.dsc.json

```
{
  "$schema": "https://schemas.microsoft.com/dsc/2024/04/config.schema.json",
  "metadata": { "description": "ProStateKit baseline" },
  "resources": [
    {
      "name": "Baseline-ControlledLocal",
      "type": "Microsoft.Windows/Group",
      "properties": {
        "groupName": "ProStateKit-Controlled",
        "ensure": "Present"
      }
    },
    {
      "name": "EnableMulticast",
      "type": "Microsoft.Windows/Registry",
      "properties": {
        "keyPath": "HKLM\\SOFTWARE\\Policies\\...",
        "valueName": "EnableMulticast",
        "valueData": 0,
        "ensure": "Present"
      }
    },
    {
      "name": "BaselineMarkerFile",
      "type": "Microsoft.Windows/File",
      "properties": {
        "destinationPath": "C:\\ProgramData\\ProStateKit\\Baseline\\baseline-applied.txt",
        "ensure": "Present"
      }
    }
  ]
}
```

Normalized Evidence Schema

`wrapper.result.json` — all fields written on every run. Automation reads this. Raw DSC output remains available alongside it.

Field	Description
<code>schemaVersion</code>	Evidence schema version
<code>operationId</code>	Unique ID for this operation
<code>runId</code>	Unique ID for this run instance
<code>mode</code>	Detect Remediate ValidateBundle
<code>plane</code>	Local Intune ConfigMgr CI
<code>bundle</code>	Bundle version and hash
<code>runtime</code>	DSC runtime path, version, hash
<code>config</code>	Config path and hash
<code>startedAt</code>	ISO 8601 run start time
<code>endedAt</code>	ISO 8601 run end time
<code>durationMs</code>	Run duration in milliseconds

Field	Description
<code>dscVersion</code>	DSC CLI version string
<code>compliant</code>	Boolean compliance decision
<code>classification</code>	Compliant NonCompliant Failed
<code>overall</code>	Aggregated resource result
<code>resources</code>	Per-resource result array
<code>reboot</code>	rebootRequired signal and marker path
<code>exitDecision</code>	Exit code and reason string
<code>evidencePath</code>	Absolute path to evidence folder
<code>errors</code>	Error array with codes and messages
<code>warnings</code>	Warning array for non-fatal issues

Runtime Distribution Checklist

1

Bundle or Govern

Bundle a pinned DSC ZIP with the payload *or* deploy DSC as a managed prerequisite to an approved installed path. Choose one; document the decision.

2

Validate on Every Run

Record runtime source, version, path, expected hash, and observed hash in evidence. Validation happens before any DSC invocation.

3

Gate LabLatest

LabLatest is for explicit compatibility testing. Production shims reject it at the entry point before reaching the Runner.

4

No Silent Downloads

No live download of `dsc.exe` during a production endpoint run. The binary must be present and validated before the run starts.

Intune Remediations Requirements

Script Paths

```
planes/intune/Detect-ProStateKit.ps1
planes/intune/Remediate-ProStateKit.ps1
```

Behavioral Requirements

Requirement	Value
Detection exit triggers remediation	Exit 1
Output size limit	Stay under portal limit — short summary only
Runtime mode	PinnedBundle
LabLatest	Rejected by production shim
Full proof location	On-disk evidence folder
Common entry point	src/Invoke-ProStateKit.ps1

Design Notes

Intune Remediations scripts have a short output window. The shim writes a human-readable summary to stdout for the portal and routes the full structured evidence to disk. Never write raw DSC JSON to Intune script output.

 LabLatest must be rejected in the production shim — not just discouraged. The entry point checks RuntimeMode before proceeding.

ConfigMgr Conversion Checklist

01

Deploy the Bundle

Use Application or Package content deployment to place the bundle on target endpoints. Use application detection to prove bundle presence by manifest version and hash.

03

Remediation Script

Use `planes/configmgr/Remediate-ProStateKit.ps1`. Validate remediation behavior matches compliance rule expectations.

02

Discovery Script

Use `planes/configmgr/Discover-ProStateKit.ps1`. Validate the discovery output type. Script must output `Compliant` or `NonCompliant` as a string.

04

Run Context

Configure run context, 64-bit behavior, log location, and client settings. Preserve the same evidence schema as all other planes.

Reboot Continuation Options

Strategy	Description	Requirements
Plane-managed restart and rerun	Preferred. Execution plane owns restart policy, notification, and retry cadence.	Plane supports post-reboot compliance re-evaluation
Marker-based detection	Runner writes <code>reboot.marker.json</code> . Plane reads marker on next evaluation cycle.	Marker path is accessible to discovery script
Scheduled continuation	A scheduled task triggers a post-reboot Detect run.	Signing, TTL, cleanup, and audit requirements must be addressed

 The Runner writes and clears `reboot.marker.json` based on normalized proof. Marker presence alone is not evidence of compliance — a post-reboot Detect run is always required to prove the endpoint reached desired state.

Secrets Scenarios

Demo Posture

The demo uses no real secrets. `src/tools/SecretHelper.ps1` is a fail-closed scaffold — it does not store or retrieve real values. Remove or replace before any production adaptation.

Production Patterns

Platform Identity

Managed identity or device credential. No embedded values in config or script.

Resource-Native Secure Input

Resources that accept secure strings natively. Execution context must support this.

DSC secret()

Requires explicit execution context and resource support. Review notes before use.

Evidence Never Stores Values

Evidence records references and outcomes. Secret values must not appear in any evidence artifact.

Validation Commands Must Be Repeatable

Run these locally before every PR and in CI on every push. The gate is only useful when it is consistent.

```
# Full validation with pre-commit skipped for local runs
npm run validate -- -SkipPreCommit
```

```
# Markdown lint only
npm run lint:md
```

```
# Pre-commit hooks across all files
pre-commit run --all-files
```

```
# Pester test suite
Invoke-Pester -Path tests/PowerShell -Output Detailed
```

```
# Schema lint
pwsh -File src/tools/Invoke-SchemaLint.ps1
```

```
# Release readiness gate (run before promotion)
tools/Test-ReleaseReadiness.ps1
```

 These commands are the canonical validation gate. If a command is not in this list, it is not part of the gate. Adding new checks here makes them first-class requirements.

Agentic Workflow Guardrails

Permitted Scope

- Issue triage and branch setup
- Small-scope edits within defined boundaries
- Test scaffolding and documentation updates
- Pull request drafting with CI rerun
- Bundle production after human review approval
- Lab evidence capture under supervision

Hard Constraints

- No secrets in any artifact
- No untrusted command execution
- No hidden telemetry
- No weakened security constraints
- No green result without evidence
- No production rollout without human approval

⊗ Agents accelerate authoring. CI owns the gate. Humans own correctness, safety, and production decisions.

Azure Machine Configuration Boundary

What Machine Configuration Provides

Azure Machine Configuration can audit and configure OS settings through Azure Policy and Arc governance paths. It supports both audit-only and audit-and-configure assignment modes with built-in reporting through the Azure portal.

 ProStateKit does not include a Machine Configuration package. That requires a completed, tested package build.

Before Implementing, Ensure:

- Assignment model and targeting
- Evaluation cadence and triggers
- Remediation behavior and scope
- Reporting surface and portal integration
- Evidence schema and retention
- Resource and OS coverage

Server and Linux Keep Their Own Planes

Before adapting ProStateKit to a server or Linux target, answer each question for that platform explicitly.

Question	Options for Server / Linux
What declares state?	DSC v3 config doc, Ansible playbook, Chef recipe, other
What tests state?	DSC config test, native check, CI assertion
What sets state?	DSC config set, idempotent script, configuration tool
What captures evidence?	Runner, CI artifact, log aggregator
What translates exits?	Runner shim, CI pipeline step
What owns reboots?	Arc, ConfigMgr, systemd, maintenance window
What owns reporting?	Azure Policy, ConfigMgr, custom dashboard

macOS Stays Native-First

Primary macOS Management Surfaces

- Apple MDM / Declarative Device Management (DDM)
- Jamf Pro or Jamf Protect
- Homebrew Bundle for package state
- nix-darwin or Home Manager for system configuration
- Internal tooling with audit integration

DSC v3 on macOS

DSC v3 can run on macOS. That does not make it the default macOS management plane. A credible macOS DSC story requires:

- Runner packaging for macOS execution context
- Sufficient resource coverage for the target state
- Identity and signing for production deployment
- Evidence schema compatible with macOS paths
- Rollback strategy

📌 Start with native controls. Reach for DSC v3 only when the native surface cannot express the requirement with equivalent proof.

Software Installation Boundary

Good Fits for DSC v3

Declared Package Presence

Pinned source, internal feed, specific version floor

Deterministic Detection

Idempotent install behavior with explicit exit translation

Explicit Ownership

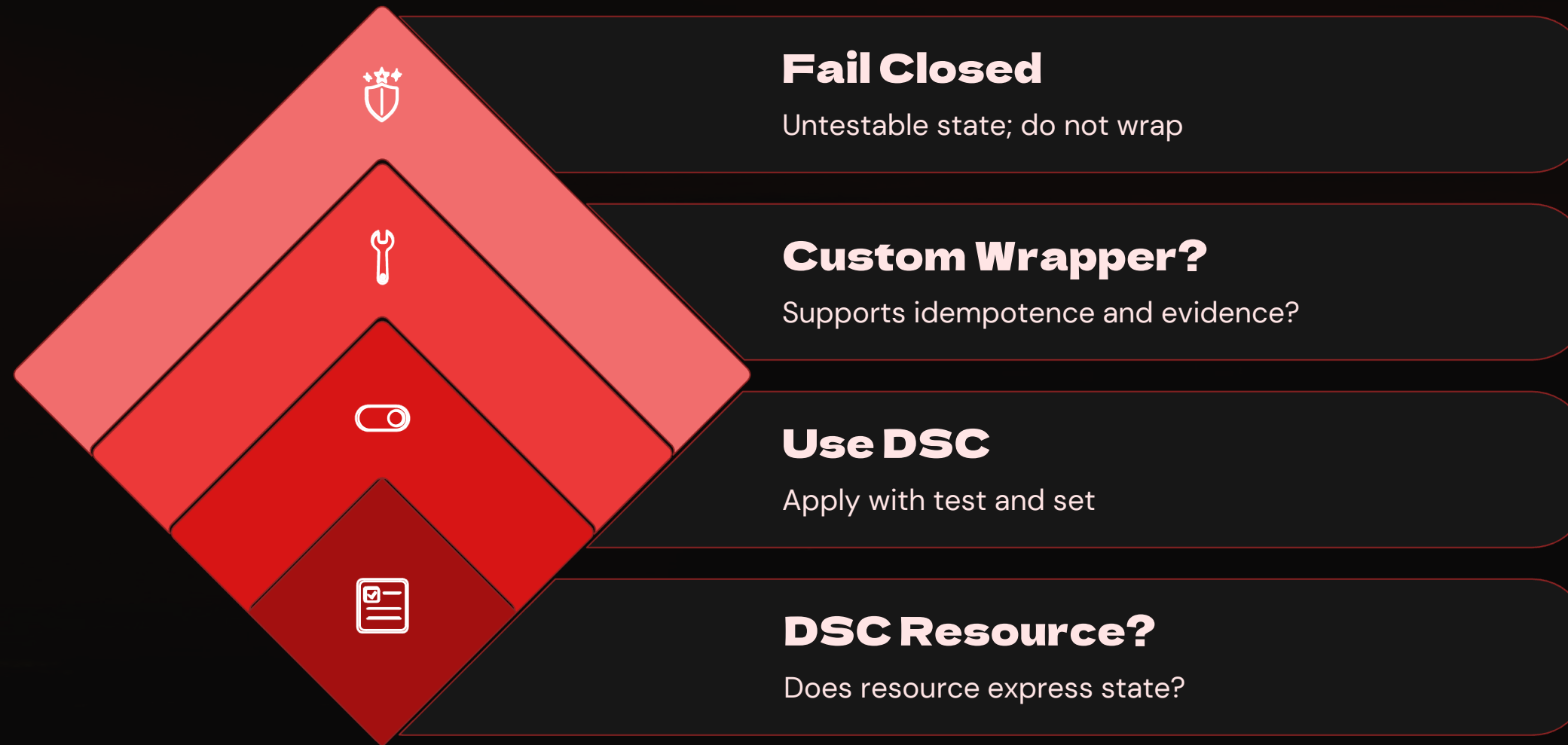
Clear accountable team and evidence capture on every run

Keep These in App Deployment Tools

- Emergency patching and out-of-band updates
- Arbitrary internet installs
- User-facing experience and notifications
- Broad catalog distribution and supersedence
- Complex restart choreography
- License and entitlement management

 A DSC resource that performs arbitrary internet package installs is an unmanaged package manager. That recreates script fatigue in a new wrapper.

Resource Gaps Need a Fail-Closed Decision



Use a DSC resource when it cleanly expresses the desired state and supports both test and set behavior. A resource that only sets state — without a reliable test — cannot prove compliance.

A fragile custom wrapper that bypasses idempotence, evidence, or fail-closed rules recreates script fatigue. If the wrapper cannot prove the endpoint reached the declared state, it is not a solution.

ProStateKit Reference Map

Concept	File / Path	Purpose
Public entry point	src/Invoke-ProStateKit.ps1	All execution planes call this
Common runner	src/runner/Runner.ps1	DSC invocation, parsing, evidence, exits
Intune shims	planes/intune/	Detect and Remediate for Intune
ConfigMgr shims	planes/configmgr/	Discover and Remediate for ConfigMgr
Local / lab shims	planes/local/	Preflight and local execution
YAML baseline	configs/baseline.dsc.yaml	Human-authored desired state
JSON baseline	configs/generated/baseline.dsc.json	Generated for schema validation
Evidence schema	schemas/wrapper-result.schema.json	Normalized evidence contract
Demo runbook	docs/runbooks/demo-runbook.md	Repeatable demo and rehearsal guide
Sample evidence	evidence/sample/	Reference compliant and noncompliant shapes

```
Administrator: Command Prompt

C:\Users\flesniak\Downloads\DSC-3.2.0-aarch64-pc-windows-msvc>dsc resource list --?
error: unexpected argument '--?' found

  tip: to pass '--?' as a value, use '-- --?'

Usage: dsc resource list [OPTIONS] [RESOURCE_NAME]

For more information, try '--help'.

C:\Users\flesniak\Downloads\DSC-3.2.0-aarch64-pc-windows-msvc>dsc resource list

Type                Kind      Version  Capabilities  RequireAdapter  Description
-----
Microsoft.Adapter/PowerShell      Adapter    0.1.0    gs-t-e-      Resource adapter to classic DSC Powershell resources.
Microsoft.Adapter/WindowsPowerShell Adapter    0.1.0    gs-t-e-      Resource adapter to classic DSC Powershell resources in Windows PowerShell.
Microsoft.DSC.Debug/Echo          Resource   1.0.0    gs-t-e-      Echo resource for testing and debugging purposes
Microsoft.DSC.Transitional/PowerShellScript Resource   0.1.0    gs-t---      Enable running PowerShell 7 scripts inline
Microsoft.DSC.Transitional/RunCommandOnSet Resource   0.1.0    gs-----      Takes a single-command line to execute on DSC set operation
Microsoft.DSC.Transitional/WindowsPowerShellScript Resource   0.1.0    gs-t---      Enable running Windows PowerShell 5.1 scripts inline
Microsoft.DSC/Assertion           Group      0.1.0    gs-t---      `test` will be invoked for all resources in the supplied configuration.
Microsoft.DSC/Group               Group      0.1.0    gs-t---      All resources in the supplied configuration is treated as a group.
Microsoft.DSC/Include             Importer    0.1.0    gs-t---      Allows including a configuration file with optional parameter file.
Microsoft.DSC/PowerShell          Adapter    0.1.0    gs-t-e-      Resource adapter to classic DSC Powershell resources.
Microsoft.PowerShell/Profile       Resource   0.1.0    gs---e-      Manage PowerShell profiles.
Microsoft.Windows/FeatureOnDemandList Resource    0.1.0    gs---e-      Manage Windows Features on Demand (capabilities) using the DISM API.
Microsoft.Windows/FirewallRuleList Resource    0.1.0    gs---e-      Manage Windows Firewall rules using the netfw.h APIs.
Microsoft.Windows/OptionalFeatureList Resource    0.1.0    gs---e-      Manage Windows Optional Features using the DISM API.
Microsoft.Windows/RebootPending   Resource    0.1.0    g-----      Returns info about pending reboot.
Microsoft.Windows/Registry        Resource    1.0.0    gs--d--      Manage Windows Registry keys and values
Microsoft.Windows/Service         Resource    0.1.0    gs---e-      Manage Windows services
Microsoft.Windows/UpdateList      Resource    0.1.0    gs---e-      Query Windows Update information for specific updates.
Microsoft.Windows/WindowsPowerShell Adapter     0.1.0    gs-t-e-      Resource adapter to classic DSC Powershell resources in Windows PowerShell.
Microsoft.Windows/WMI             Adapter     1.0.0    gs-----      Resource adapter to WMI resources.
Microsoft.WinGet/AdminSettings     Resource    1.28.240 gs-t-e-      Allows management of administrator settings via the DSC v3 command line interface protocol. See the help link fo...
Microsoft.WinGet/Package          Resource    1.28.240 gsxt-e-      Allows management of package state via the DSC v3 command line interface protocol. See the help link for details...
Microsoft.WinGet/Source           Resource    1.28.240 gsxt-e-      Allows management of source configuration via the DSC v3 command line interface protocol. See the help link for ...
Microsoft.WinGet/UserSettingsFile Resource    1.28.240 gsxt-e-      Allows management of user settings file via the DSC v3 command line interface protocol. See the help link for de...
```

Windows Resources

Type	Kind
Microsoft.Adapter/PowerShell	Adapter
Microsoft.Adapter/WindowsPowerShell	Adapter
Microsoft.DSC.Debug/Echo	Resource
Microsoft.DSC.Transitional/PowerShellScript	Resource
Microsoft.DSC.Transitional/RunCommandOnSet	Resource
Microsoft.DSC.Transitional/WindowsPowerShellScript	Resource
Microsoft.DSC/Assertion	Group
Microsoft.DSC/Group	Group
Microsoft.DSC/Include	Importer
Microsoft.DSC/PowerShell	Adapter
Microsoft.PowerShell/Profile	Resource
Microsoft.Windows/FeatureOnDemandList	Resource
Microsoft.Windows/FirewallRuleList	Resource
Microsoft.Windows/OptionalFeatureList	Resource
Microsoft.Windows/RebootPending	Resource
Microsoft.Windows/Registry	Resource
Microsoft.Windows/Service	Resource
Microsoft.Windows/UpdateList	Resource
Microsoft.Windows/WindowsPowerShell	Adapter
Microsoft.Windows/WMI	Adapter
Microsoft.WinGet/AdminSettings	Resource
Microsoft.WinGet/Package	Resource
Microsoft.WinGet/Source	Resource
Microsoft.WinGet/UserSettingsFile	Resource

Description
Resource adapter to classic DSC Powershell resources.
Resource adapter to classic DSC Powershell resources in Windows PowerShell.
Echo resource for testing and debugging purposes
Enable running PowerShell 7 scripts inline
Takes a single-command line to execute on DSC set operation
Enable running Windows PowerShell 5.1 scripts inline
`test` will be invoked for all resources in the supplied configuration.
All resources in the supplied configuration is treated as a group.
Allows including a configuration file with optional parameter file.
Resource adapter to classic DSC Powershell resources.
Manage PowerShell profiles.
Manage Windows Features on Demand (capabilities) using the DISM API.
Manage Windows Firewall rules using the netfw.h APIs.
Manage Windows Optional Features using the DISM API.
Returns info about pending reboot.
Manage Windows Registry keys and values
Manage Windows services
Query Windows Update information for specific updates.
Resource adapter to classic DSC Powershell resources in Windows PowerShell.
Resource adapter to WMI resources.
Allows management of administrator settings via the DSC v3 command line interface protocol. See the help link fo...
Allows management of package state via the DSC v3 command line interface protocol. See the help link for details...
Allows management of source configuration via the DSC v3 command line interface protocol. See the help link for ...
Allows management of user settings file via the DSC v3 command line interface protocol. See the help link for de...

Windows Resources

Type	Kind	Version	Capabilities	RequireAdapter	Description
DSC.PackageManagement/Brew	Resource	0.1.0	gs---e-		DSC resource to manage Homebrew packages
Microsoft.Adapter/PowerShell	Adapter	0.1.0	gs-t-e-		Resource adapter to classic DSC Powershell resources.
Microsoft.DSC.Debug/Echo	Resource	1.0.0	gs-t-e-		Echo resource for testing and debugging purposes
Microsoft.DSC.Transitional/PowerShellScript	Resource	0.1.0	gs-t---		Enable running PowerShell 7 scripts inline
Microsoft.DSC.Transitional/RunCommandOnSet	Resource	0.1.0	gs-----		Takes a single-command line to execute on DSC set operation
Microsoft.DSC/Assertion	Group	0.1.0	gs-t---		'test' will be invoked for all resources in the supplied configuration.
Microsoft.DSC/Group	Group	0.1.0	gs-t---		All resources in the supplied configuration is treated as a group.
Microsoft.DSC/Include	Importer	0.1.0	gs-t---		Allows including a configuration file with optional parameter file.
Microsoft.DSC/PowerShell	Adapter	0.1.0	gs-t-e-		Resource adapter to classic DSC Powershell resources.
Microsoft.OpenSSH.SSHD/sshd_config	Resource	0.1.0	gs---e-		Manage SSH Server Configuration
Microsoft.OpenSSH.SSHD/Subsystem	Resource	0.1.0	gsx----		Manage SSH Server Subsystem keyword (single entry)
Microsoft.OpenSSH.SSHD/SubsystemList	Resource	0.1.0	gs-----		Manage SSH Server Subsystem keyword (accepts multiple entries)
Microsoft/OSInfo	Resource	0.1.0	g----e-		Returns information about the operating system.

macOS Resources

Type	Kind	Version
DSC.PackageManagement/Brew	Resource	0.1.0
Microsoft.Adapter/PowerShell	Adapter	0.1.0
Microsoft.DSC.Debug/Echo	Resource	1.0.0
Microsoft.DSC.Transitional/PowerShellScript	Resource	0.1.0
Microsoft.DSC.Transitional/RunCommandOnSet	Resource	0.1.0
Microsoft.DSC/Assertion	Group	0.1.0
Microsoft.DSC/Group	Group	0.1.0
Microsoft.DSC/Include	Importer	0.1.0
Microsoft.DSC/PowerShell	Adapter	0.1.0
Microsoft.OpenSSH.SSHD/sshd_config	Resource	0.1.0
Microsoft.OpenSSH.SSHD/Subsystem	Resource	0.1.0
Microsoft.OpenSSH.SSHD/SubsystemList	Resource	0.1.0
Microsoft/OSInfo	Resource	0.1.0

Description

DSC resource to manage Homebrew packages
Resource adapter to classic DSC Powershell resources.
Echo resource for testing and debugging purposes
Enable running PowerShell 7 scripts inline
Takes a single-command line to execute on DSC set operation
'test' will be invoked for all resources in the supplied configuration.
All resources in the supplied configuration is treated as a group.
Allows including a configuration file with optional parameter file.
Resource adapter to classic DSC Powershell resources.
Manage SSH Server Configuration
Manage SSH Server Subsystem keyword (single entry)
Manage SSH Server Subsystem keyword (accepts multiple entries)
Returns information about the operating system.

macOS Resources